MODELOS Y SISTEMAS

Proyecto integrador – Contenidos de 5.º y 7.º año

Nombre y apellido: Tomas Videla

Curso y division: 7 ° 2da

Docente: Gian Franco

Fecha entrega: 13/11/2026

Bibliografia: Senn, James A. (1992). Análisis y Diseño de Sistemas de Información. McGraw-Hill.

Pressman, Roger S. (2010). Ingeniería del Software: Un Enfoque Práctico. McGraw-Hill.

Kendall, Kenneth E. & Kendall, Julie E. (2011). Análisis y Diseño de Sistemas. Pearson.

Taha, Hamdy A. (2012). Investigación de Operaciones. Pearson.

Booch, Grady; Rumbaugh, James & Jacobson, Ivar (2006). El Lenguaje Unificado de Modelado (UML): Guía del Usuario. Addison-Wesley.

Deitel, Paul J. & Deitel, Harvey M. (2017). C# para Programadores. Anaya Multimedia.

Normas IRAM y Estándares Administrativos.

Chacon, Scott & Straub, Ben (2014). Pro Git. Apress / Documentación de GitHub.

Introducción

Modelos y Sistemas es una materia que estudia cómo comprender problemas, representar información y diseñar soluciones mediante sistemas. Para desarrollar un sistema informático no alcanza con escribir código: primero es necesario conocer qué necesita el usuario, analizar los datos, organizar los procesos, planificar el trabajo, diseñar una solución y documentar lo realizado. En 5.º año se trabajan herramientas de análisis y modelado; en 7.º año esos conocimientos se relacionan con la programación orientada a objetos, las bases de datos, la gestión de proyectos y la documentación.

5.º AÑO – Análisis y modelado de sistemas

1. Herramientas de diagramación

Las herramientas de diagramación permiten representar visualmente el funcionamiento de un sistema. Ayudan a ordenar ideas, detectar errores, explicar procesos y comunicar cómo se relacionan los datos y las actividades. Para que los diagramas puedan ser entendidos por otras personas, se utilizan símbolos y convenciones comunes.

Diagrama de flujo de datos (DFD)

El diagrama de flujo de datos muestra cómo circula la información dentro de un sistema. No se centra en el orden exacto de las instrucciones, sino en el origen, transformación, almacenamiento y destino de los datos. Sus elementos principales son:

- **Entidad externa:** persona, área u otro sistema que envía información o recibe resultados, pero que se encuentra fuera del sistema analizado.
- **Proceso:** actividad que transforma datos de entrada en información de salida. Se representa con un nombre que indique una acción, por ejemplo, “Registrar venta”.
- **Flujo de datos:** flecha que indica qué información se mueve y hacia dónde se dirige.
- **Almacén de datos:** lugar donde la información queda guardada para utilizarse más adelante, como un archivo o una base de datos.

Un DFD puede realizarse en distintos niveles. El diagrama de contexto muestra al sistema como un único proceso y sus intercambios con el exterior. Los niveles siguientes descomponen ese proceso en partes más detalladas.

Diagrama de control

El diagrama de control representa las condiciones, señales o eventos que regulan el comportamiento de un sistema. Permite mostrar qué ocurre cuando se cumple una condición, cuándo se activa una acción y qué camino sigue el proceso ante diferentes situaciones. Es útil en sistemas que responden a eventos, alarmas, autorizaciones o cambios de estado. A diferencia del DFD, que se concentra en el movimiento de los datos, el diagrama de control se enfoca en qué controla o desencadena las acciones.

Diccionario de datos

El diccionario de datos es un documento que reúne las definiciones de todos los datos que aparecen en un sistema o en sus diagramas. Para cada dato puede especificar nombre, significado, tipo (texto, número, fecha, etc.), longitud, formato, valores permitidos, origen y reglas de validación. Por ejemplo, el dato “DNI” puede definirse como numérico o texto, con una cantidad determinada de caracteres y sin valores repetidos. Su objetivo es evitar confusiones, mantener nombres consistentes y facilitar el diseño de archivos y bases de datos.

Lógica de los procesos

La lógica de un proceso explica paso a paso qué debe hacer el sistema para cumplir una tarea. Define qué datos recibe, qué operaciones realiza, qué condiciones evalúa y qué resultado produce. Puede incluir cálculos, comparaciones, validaciones, decisiones y repeticiones. Por ejemplo, para registrar una inscripción, el sistema puede comprobar si el alumno existe, verificar si hay vacantes y luego guardar la inscripción o informar por qué no se pudo realizar. Una lógica clara ayuda a evitar errores y permite programar el proceso correctamente.

Almacenamiento de datos

El almacenamiento consiste en conservar datos para consultarlos, modificarlos o utilizarlos posteriormente. La información puede guardarse en archivos, hojas de cálculo o bases de datos. En los sistemas de información se busca que los datos estén organizados, disponibles cuando se necesitan y protegidos contra pérdidas o accesos no autorizados. También es importante mantener su integridad, evitar duplicaciones innecesarias y realizar copias de seguridad. La elección del almacenamiento depende de la cantidad de información, la frecuencia de uso y las necesidades del sistema.

Convenciones

Las convenciones son acuerdos o reglas que indican cómo deben nombrarse y representarse los elementos de un modelo. Incluyen el uso de símbolos, flechas, nombres, abreviaturas, niveles de detalle y formas de conexión. Por ejemplo, en un diagrama de flujo se utilizan símbolos específicos para el inicio, los procesos y las decisiones. Respetar las convenciones permite que diferentes personas interpreten el diagrama de la misma manera y facilita su lectura y mantenimiento.

2. Diagramas utilizados en el análisis

Diagrama de transición de estados

Este diagrama representa los estados posibles de un objeto o sistema y los eventos que provocan el paso de un estado a otro. Un estado describe una situación en la que se encuentra el elemento durante un período. Por ejemplo, un pedido puede estar “Pendiente”, “Preparándose”, “Enviado” o “Entregado”. Una acción o evento, como confirmar el pago, puede provocar una transición. El diagrama permite comprender el comportamiento del sistema y comprobar que estén contempladas las distintas situaciones posibles.

Diagrama entidad-relación (DER)

El diagrama entidad-relación representa los datos que necesita una organización y la manera en que se vinculan. Una entidad es algo sobre lo que se desea guardar información, como Cliente, Producto o Pedido. Los atributos son sus características, por ejemplo nombre, precio o fecha. Las relaciones indican cómo se conectan las entidades. Las cardinalidades más comunes son uno a uno (1:1), uno a muchos (1:N) y muchos a muchos (N:M). Este modelo sirve como base para diseñar una base de datos organizada.

3. Técnicas para el análisis de sistemas

El análisis de sistemas consiste en investigar una situación o problema para conocer cómo funciona actualmente y qué solución se necesita. El analista reúne información mediante entrevistas, encuestas, observación directa y revisión de documentos. Luego identifica usuarios, necesidades, datos, procesos, restricciones y resultados esperados. Un buen análisis evita desarrollar un sistema que no responda al problema real.

Desarrollo, control y seguimiento del proyecto

El desarrollo del proyecto es la realización de las actividades planificadas para construir la solución. El control consiste en comparar lo que se está haciendo con lo que se había previsto, revisando calidad, tiempo, costos y cumplimiento de objetivos. El seguimiento es la observación periódica del avance: se registran tareas terminadas, tareas pendientes, dificultades y cambios. Estas actividades permiten detectar retrasos o problemas a tiempo y tomar medidas para corregirlos.

Prototipo

Un prototipo es una representación inicial y simplificada del sistema que se quiere construir. Puede ser un dibujo de pantallas, un formulario navegable o una versión con algunas funciones básicas. Se presenta a los usuarios para conocer sus opiniones, comprobar si la idea se entiende y descubrir necesidades que no se habían considerado. Luego se realizan modificaciones. El prototipo no siempre es el producto final: su función principal es probar y mejorar la propuesta antes de desarrollarla por completo.

Ciclo de vida de un sistema

El ciclo de vida es el conjunto de etapas por las que pasa un sistema desde que se identifica una necesidad hasta que deja de utilizarse o es reemplazado. Una forma habitual de organizarlo es:

- **Planificación:** se define el problema, los objetivos, los recursos, los tiempos y la viabilidad.
- **Análisis:** se estudian las necesidades de los usuarios y los requisitos del sistema.
- **Diseño:** se determina cómo será la solución, incluyendo datos, procesos, pantallas y estructura.
- **Desarrollo:** se construye el sistema mediante programación y configuración.
- **Pruebas:** se comprueba que funcione correctamente y se corrigen errores.
- **Implementación:** se instala y se pone en funcionamiento para los usuarios.
- **Mantenimiento:** se corrigen fallas, se adapta el sistema y se incorporan mejoras.

Estas etapas pueden organizarse de distintas maneras según la metodología utilizada. En algunos proyectos se realizan de forma secuencial y en otros se repiten en ciclos de mejora.

4. Números al azar

Los números al azar se utilizan para representar situaciones impredecibles o variables en simulaciones, experimentos, juegos, estadísticas y modelos matemáticos. Por ejemplo, pueden servir para simular la llegada de clientes a un comercio o elegir casos de una muestra. En informática es habitual generar números pseudoaleatorios: secuencias producidas por una fórmula o algoritmo que parecen aleatorias, aunque se calculan a partir de un estado inicial.

Funciones generadoras de números aleatorios

Una función generadora es un procedimiento que produce números dentro de un rango determinado. Muchas funciones permiten indicar una semilla (seed), que es el valor inicial del cálculo. Si se utiliza la misma semilla y el mismo algoritmo, se obtiene la misma secuencia, lo cual resulta útil para repetir pruebas. Para que una secuencia sea adecuada debe distribuirse de forma razonable y no presentar patrones fáciles de reconocer.

Métodos manuales y algoritmos

Los métodos manuales utilizan recursos como tablas impresas de números aleatorios, sorteos o mecanismos físicos. Son fáciles de aplicar en actividades pequeñas o educativas, aunque pueden resultar lentos cuando se necesitan muchos valores. Los algoritmos son conjuntos de instrucciones matemáticas que generan números mediante una computadora. Un ejemplo clásico es el método congruencial, que calcula cada valor a partir del anterior usando operaciones de multiplicación, suma y módulo. Los algoritmos son rápidos y permiten repetir experimentos, pero la secuencia depende de la calidad del método.

Comparación de los métodos

Los métodos manuales pueden ser útiles cuando se trabaja con pocos datos y se desea una selección sencilla. Los algoritmos permiten producir grandes secuencias en poco tiempo, automatizar tareas y reproducir resultados usando la misma semilla. Sin embargo, no todos los generadores tienen la misma calidad: algunos pueden repetir patrones o distribuir mal los valores. Por eso, el método se elige según el objetivo y el nivel de precisión requerido.

5. Teoría de los modelos

Un modelo es una representación simplificada de una parte de la realidad. Se construye para estudiar una situación, comprender cómo se relacionan sus elementos, probar alternativas o anticipar resultados. Un modelo no contiene todos los detalles del mundo real: conserva los que son importantes para el objetivo del análisis.

Tablas de decisión

Una tabla de decisión organiza condiciones y acciones en forma de tabla. Cada columna representa una combinación posible de condiciones y muestra qué acción corresponde realizar. Por ejemplo, para otorgar un descuento se pueden considerar el tipo de cliente y el monto de compra. La tabla permite revisar todas las combinaciones, ordenar reglas y detectar casos sin respuesta o reglas contradictorias.

Árboles de decisión

Un árbol de decisión representa gráficamente una serie de preguntas, condiciones o elecciones mediante ramas. Se comienza en un punto inicial y, según la respuesta o condición, se sigue una rama hasta llegar a una acción o resultado. Es útil para visualizar caminos posibles y explicar decisiones complejas. Mientras la tabla permite comparar combinaciones de manera compacta, el árbol muestra el recorrido de cada alternativa.

Abstracción de datos

La abstracción de datos consiste en seleccionar la información relevante para un objetivo y dejar de lado detalles que no son necesarios. Por ejemplo, para un sistema de biblioteca puede ser importante registrar título, autor y disponibilidad de un libro, pero no el color de las paredes del edificio. Abstraer permite simplificar la realidad, organizar los datos y concentrarse en lo que el sistema necesita resolver.

Objetivos del uso de modelos matemáticos

Los modelos matemáticos utilizan números, variables, operaciones y ecuaciones para representar relaciones de una situación real. Se utilizan para explicar fenómenos, calcular resultados, comparar alternativas, optimizar recursos, evaluar riesgos y realizar predicciones. Por ejemplo, un comercio puede modelar sus ingresos y gastos para estimar ganancias o calcular cuántos productos necesita vender para cubrir sus costos.

Clasificación de modelos

Los modelos se pueden clasificar de distintas maneras según qué representan y cómo funcionan:

- **Físicos:** representaciones materiales o a escala, como una maqueta.
- **Conceptuales o gráficos:** expresan ideas y relaciones mediante textos, esquemas o diagramas.
- **Matemáticos:** utilizan fórmulas, variables y ecuaciones.
- **Estáticos:** representan una situación en un momento determinado.
- **Dinámicos:** muestran cómo cambia una situación a lo largo del tiempo.
- **Determinísticos:** con los mismos datos de entrada producen el mismo resultado.
- **Probabilísticos:** incluyen el azar o la probabilidad y pueden producir distintos resultados.
- **Descriptivos:** explican o representan cómo es una situación.
- **Predictivos:** se utilizan para estimar qué podría suceder.

Un mismo modelo puede pertenecer a más de una categoría. Por ejemplo, puede ser matemático, dinámico y probabilístico al mismo tiempo.

Predicción de los resultados

Predecir es estimar qué puede ocurrir utilizando un modelo y determinados datos de entrada. Primero se definen las variables y las relaciones, luego se introducen datos y se calculan los resultados. Es importante comparar las predicciones con datos reales para comprobar si el modelo representa

adecuadamente la situación. Una predicción no es una certeza: puede verse afectada por datos incompletos, cambios en el contexto o supuestos incorrectos.

6. Diseño de documentación y manuales de sistemas y usuarios

La documentación reúne información sobre el análisis, diseño, funcionamiento, instalación y uso de un sistema. Es necesaria para que otras personas puedan comprenderlo, utilizarlo, mantenerlo y modificarlo sin depender únicamente de quienes lo desarrollaron.

Técnicas para elaborar la documentación de las tareas

Para documentar una tarea se debe identificar su objetivo y describirla de manera ordenada. Conviene registrar qué actividad se realiza, quién es responsable, qué información o materiales necesita, cuáles son los pasos, qué resultado debe obtenerse y qué controles deben efectuarse. Se pueden utilizar descripciones escritas, listas de pasos, tablas, capturas de pantalla, diagramas y ejemplos. La documentación debe ser clara, precisa, completa y mantenerse actualizada.

Lineamientos generales para confeccionar un manual

Un manual debe tener un título claro, propósito, destinatarios, índice y secciones ordenadas. Debe utilizar lenguaje sencillo, explicar los términos técnicos necesarios y presentar los procedimientos en el orden en que deben realizarse. Es conveniente incorporar imágenes, capturas o ejemplos cuando faciliten la comprensión. También debe indicar requisitos, advertencias y formas de resolver problemas frecuentes. Para evitar confusiones, se deben incluir fecha, número de versión y responsable de la actualización.

Conformación y organización de la documentación

La documentación debe organizarse para que la información pueda encontrarse fácilmente. Puede dividirse por tipo (técnica o de usuario), por etapa del proyecto o por módulo del sistema. Es conveniente mantener un índice, nombres de archivos coherentes, numeración de páginas, fechas y versiones. También se deben establecer responsables de actualización y conservar copias de seguridad. Una documentación desordenada o desactualizada puede provocar errores durante el uso y mantenimiento del sistema.

7.º AÑO – Programación, diseño y gestión de sistemas

1. Programación orientada a objetos (POO)

La programación orientada a objetos es un paradigma de programación que organiza el código en objetos. Cada objeto reúne datos (atributos) y operaciones (métodos) relacionadas. Este enfoque permite dividir un programa en partes, reutilizar código y representar elementos del problema de una manera ordenada. Sus conceptos principales son clase, objeto, encapsulamiento, herencia y polimorfismo.

Clase y objeto

Una clase es una plantilla o modelo que define qué características y comportamientos tendrán sus objetos. Un objeto es una instancia concreta creada a partir de esa clase. Por ejemplo, la clase “Alumno” puede definir los atributos nombre y legajo y el método “mostrarDatos”. Un objeto sería un alumno particular con su propio nombre y legajo. Muchos objetos pueden crearse a partir de una misma clase.

Atributos y métodos

Los atributos son variables que guardan el estado o las características de un objeto, como nombre, edad o precio. Los métodos son funciones o procedimientos que indican qué acciones puede realizar,

como calcular un total, actualizar un dato o mostrar información. En conjunto, atributos y métodos describen las propiedades y el comportamiento de los objetos.

Creación de clases

Para crear una clase se define un nombre y se determinan sus atributos y métodos. Luego se establece la visibilidad de cada elemento y, si corresponde, se programa un constructor. Es importante que la clase tenga una responsabilidad clara. Por ejemplo, una clase “Producto” puede contener código, descripción y precio, además de métodos para consultar o actualizar esos datos.

Paquetes

Un paquete es una forma de agrupar clases y otros elementos relacionados dentro de un proyecto. Permite ordenar el código por función o área, evitar desorganización y facilitar el mantenimiento. Por ejemplo, un sistema puede tener paquetes llamados “Modelos”, “Vistas”, “Controladores” o “Datos”, según la arquitectura utilizada.

Instancia

La instancia es el objeto concreto que se crea a partir de una clase. Al instanciar una clase se reserva espacio para el objeto y se inicializan sus datos. Cada instancia tiene sus propios valores de atributos, aunque todas comparten la estructura y los métodos definidos por la clase.

Tipos de visibilidad

La visibilidad determina desde qué partes del programa se puede acceder a atributos y métodos. Los modificadores más habituales son:

- **Public:** permite acceder al elemento desde otras clases o partes del programa.
- **Private:** limita el acceso a la propia clase. Se utiliza para proteger los datos internos.
- **Protected:** permite el acceso desde la propia clase y, generalmente, desde sus clases derivadas o hijas.

La disponibilidad y el alcance exacto de estos modificadores pueden variar según el lenguaje de programación.

Get y Set

Los métodos get y set permiten controlar el acceso a los atributos. Get devuelve o consulta el valor de un atributo; set permite asignarle un nuevo valor. Por ejemplo, un método set para la edad puede comprobar que no se ingrese un valor negativo. Se relacionan con el encapsulamiento, porque evitan que los datos internos se modifiquen sin controles.

Constructores

Un constructor es un método especial que se ejecuta cuando se crea un objeto. Su función principal es inicializar los atributos y dejar al objeto en un estado válido. Puede recibir parámetros para asignar valores desde el momento de la creación. Una clase puede tener más de un constructor si el lenguaje permite sobrecarga.

Herencia

La herencia permite crear una clase nueva a partir de otra existente. La clase hija recibe atributos y métodos de la clase padre y puede agregar características propias o redefinir algunos comportamientos. Por ejemplo, “Alumno” y “Docente” podrían derivar de una clase general “Persona”. La herencia evita repetir código, pero debe utilizarse cuando existe una relación lógica de tipo “es un”.

Listas y ArrayList

Una lista es una colección que permite guardar varios elementos en una sola estructura y recorrerlos mediante posiciones o iteraciones. ArrayList es una colección dinámica disponible en determinados lenguajes y entornos, que permite agregar o quitar elementos durante la ejecución. A diferencia de un

arreglo de tamaño fijo, puede modificar su cantidad de elementos. En algunas implementaciones acepta distintos tipos de objetos; las listas genéricas suelen ser preferibles cuando se desea restringir el tipo de dato.

Métodos add y remove

El método add agrega un elemento a la colección, generalmente al final o en una posición indicada. El método remove elimina un elemento, ya sea por su posición o por su valor, según la implementación. Por ejemplo, en una lista de alumnos, add permite incorporar un alumno nuevo y remove quitar uno que ya no debe figurar. Es necesario verificar que el elemento exista y que la posición sea válida antes de eliminarlo.

2. Diagrama entidad-relación, atributos, PK y relaciones

El diagrama entidad-relación se utiliza para diseñar la estructura de una base de datos antes de implementarla. Una entidad representa un elemento sobre el que se guarda información, como Alumno, Materia o Inscripción. Los atributos describen a la entidad: un Alumno puede tener legajo, nombre y apellido.

La clave primaria o PK (Primary Key) es el atributo, o conjunto de atributos, que identifica de manera única cada registro de una tabla. No debe repetirse y no puede quedar sin valor. Por ejemplo, el legajo puede identificar a cada alumno. Una clave foránea o FK (Foreign Key) es un campo que referencia la clave primaria de otra tabla y permite establecer vínculos.

Las relaciones expresan cómo se conectan las entidades. En una relación 1:1, un registro de una entidad se vincula con uno de otra; en 1:N, uno puede vincularse con muchos; y en N:M, varios registros de cada entidad pueden relacionarse entre sí. En una base relacional, una relación N:M suele implementarse mediante una tabla intermedia.

3. Diagrama de clases

El diagrama de clases es una herramienta de UML (Lenguaje Unificado de Modelado) que muestra la estructura de un sistema orientado a objetos. Cada clase suele representarse con un rectángulo dividido en compartimentos: nombre, atributos y métodos. También muestra relaciones entre clases, como asociación, herencia, agregación o composición. Sirve para planificar la estructura del programa antes de escribir el código y para comunicar el diseño a otros integrantes del equipo.

No debe confundirse con el diagrama entidad-relación: el DER se enfoca en los datos y sus vínculos para diseñar una base de datos; el diagrama de clases representa la estructura de las clases y objetos del software, incluyendo atributos y comportamientos.

4. ABM: alta, baja y modificación

ABM significa Alta, Baja y Modificación, y reúne operaciones básicas para administrar información. Alta es incorporar un nuevo registro; antes de guardarlo se suelen validar los datos obligatorios y comprobar que no exista una identificación repetida. Baja es quitar un registro: puede ser física, cuando se elimina de la base, o lógica, cuando se marca como inactivo para conservar el historial. Modificación permite actualizar datos existentes y debe comprobar que el registro se encuentre y que los nuevos valores sean válidos.

En muchos sistemas también se incluye la consulta o lectura de registros. Por eso se utiliza CRUD: Create (crear), Read (leer o consultar), Update (actualizar) y Delete (eliminar). ABM y CRUD son conceptos muy relacionados y aparecen en aplicaciones de gestión.

5. Cursogramas

Un cursograma es una representación gráfica de un procedimiento, especialmente de tareas administrativas y circulación de documentos o información. Muestra qué actividades se realizan, quién las lleva a cabo, en qué orden y cómo se trasladan los documentos. Su utilidad es comprender el

funcionamiento actual, detectar demoras, controles repetidos, pasos innecesarios o responsabilidades poco claras y proponer mejoras.

Elementos y técnicas para elaborar cursogramas

Para elaborar un cursograma se selecciona el procedimiento que se va a estudiar, se releva información mediante entrevistas y observación, se identifican los responsables y documentos, se ordenan las actividades y se dibuja el recorrido utilizando símbolos acordados. Finalmente se revisa el gráfico con las personas que conocen el proceso para comprobar que represente la realidad. Se debe mantener un criterio uniforme y agregar referencias cuando sean necesarias.

Símbolos: denominación y descripción

Los símbolos pueden variar de acuerdo con la norma utilizada en la institución. Entre los más frecuentes en cursogramas y diagramas de procesos se encuentran:

- **Operación:** indica una actividad que modifica, prepara o procesa algo; por ejemplo, completar un formulario.
- **Control o inspección:** representa la revisión o verificación de datos, documentos o resultados.
- **Traslado o transporte:** muestra el movimiento de un documento, material o información de un lugar o responsable a otro.
- **Demora o espera:** señala un tiempo en el que el trámite queda detenido antes de continuar.
- **Archivo o almacenamiento:** indica que un documento o dato se guarda para consultarlo o conservarlo.
- **Decisión:** representa una condición que puede derivar en distintos caminos, como “aprobado” o “rechazado”.

En un trabajo formal se debe utilizar la simbología indicada por el docente o la norma adoptada, ya que algunos símbolos y sus formas pueden cambiar entre cursogramas, diagramas de flujo y otras técnicas.

6. Diagrama de flujo

El diagrama de flujo representa gráficamente los pasos de un algoritmo o procedimiento y el orden en que se ejecutan. Utiliza símbolos conectados por flechas. Permite visualizar la secuencia, las decisiones, las repeticiones y los resultados, por lo que ayuda a planificar un programa y encontrar errores en la lógica antes de codificarlo.

Símbolos principales

- **Óvalo:** indica el inicio o el fin del proceso.
- **Rectángulo:** representa una operación o proceso, como realizar un cálculo o asignar un valor.
- **Rombo:** representa una decisión o condición, que genera caminos diferentes según la respuesta.
- **Paralelogramo:** indica la entrada o salida de datos, como leer un valor o mostrar un mensaje.
- **Flechas o líneas de flujo:** indican la dirección y el orden de los pasos.
- **Conector:** permite unir partes del diagrama cuando el recorrido continúa en otro lugar o página.

El diagrama debe tener un inicio y un fin claros, utilizar flechas correctamente y evitar cruces innecesarios. Cada decisión debe mostrar sus posibles salidas.

7. Casos de uso

Los casos de uso describen las funciones que un sistema ofrece desde el punto de vista de quienes interactúan con él. El actor es una persona, organización o sistema externo que utiliza alguna función. El caso de uso representa un objetivo, como iniciar sesión, registrar un pedido o consultar una nota. El límite del sistema indica qué funcionalidades forman parte del sistema analizado.

Además del diagrama, cada caso de uso puede describirse mediante un texto que incluya nombre, objetivo, actor principal, condiciones previas, pasos principales, alternativas y resultado final. Se utiliza

durante el análisis para identificar requisitos funcionales y comprobar que las necesidades de los usuarios estén contempladas.

8. Documentación de proyectos

La documentación de un proyecto registra la información necesaria para comprender qué se construirá, por qué se hace, cómo se desarrollará, cómo se utilizará y cómo se mantendrá. Sirve para organizar el trabajo, comunicar decisiones, controlar avances y facilitar futuras modificaciones. Una forma de organizarla en siete pasos o partes es la siguiente:

- **1. Documentación de relevamiento:** registra el problema, la situación actual, los usuarios involucrados, las entrevistas, observaciones, necesidades y requisitos detectados.
- **2. Documentación de análisis:** ordena y define los requisitos funcionales (qué debe hacer el sistema) y no funcionales (seguridad, rendimiento, facilidad de uso, entre otros), además de límites y prioridades.
- **3. Documentación de diseño:** explica cómo se organizará la solución. Puede incluir diagramas, diseño de base de datos, clases, procesos, pantallas, arquitectura y decisiones de diseño.
- **4. Documentación técnica:** está dirigida al equipo técnico e informa lenguajes, herramientas, estructura del código, componentes, configuración, conexiones, dependencias y criterios de mantenimiento.
- **5. Documentación de usuario:** explica cómo utilizar el sistema, con instrucciones, ejemplos, descripción de pantallas y solución de dificultades frecuentes.
- **6. Documentación de instalación y despliegue:** indica requisitos del entorno, pasos de instalación, configuración, publicación, migración de datos y comprobaciones para poner el sistema en funcionamiento.
- **7. Control de versiones y mantenimiento:** registra las modificaciones realizadas, sus responsables, fechas y versiones; también incluye correcciones, mejoras y actualizaciones de la documentación.

Documentación de tareas y procesos del sistema

Además de describir el programa en general, es necesario documentar las tareas y los procesos que realiza. Para cada proceso se puede indicar su finalidad, quién lo inicia, qué datos necesita, qué pasos ejecuta, qué validaciones aplica, qué resultado produce y qué ocurre si se presenta un error. Esta información puede complementarse con diagramas de flujo, casos de uso, cursogramas, capturas de pantalla y ejemplos. Documentar los procesos permite que el equipo comprenda cómo funciona el sistema y pueda mantenerlo.

Documentación técnica y documentación de usuarios

La documentación técnica está destinada a programadores, administradores y personal de soporte. Explica cómo está construido el sistema y qué se necesita para modificarlo, instalarlo o resolver fallas. La documentación de usuarios está destinada a las personas que utilizan el sistema para realizar sus tareas. Debe evitar explicaciones de código y concentrarse en instrucciones prácticas, claras y ordenadas.

Manual de usuario

El manual de usuario es un documento que enseña a utilizar una aplicación. Generalmente incluye presentación y objetivo del sistema, requisitos de acceso, forma de iniciar sesión, descripción de las pantallas, instrucciones paso a paso para cada función, mensajes de error y recomendaciones de seguridad. Debe estar escrito con palabras sencillas y acompañarse de capturas o ejemplos cuando sea posible. Su finalidad es que el usuario pueda realizar sus tareas sin necesitar ayuda constante.

9. Git y GitHub

Git es un sistema de control de versiones. Registra los cambios realizados en los archivos de un proyecto y permite consultar el historial, recuperar versiones anteriores, trabajar en ramas y combinar aportes de distintas personas. Es una herramienta muy utilizada en programación porque ayuda a organizar el trabajo y evitar que se pierdan modificaciones.

GitHub es una plataforma en línea que permite alojar repositorios Git y colaborar con otras personas. Ofrece herramientas para revisar cambios, proponer integraciones, informar errores y organizar tareas. Git es el sistema que controla las versiones; GitHub es un servicio que permite guardar repositorios y colaborar a través de Internet.

Comandos básicos de Git

- **git init:** crea o inicializa un repositorio Git en una carpeta.
- **git status:** muestra qué archivos fueron modificados, agregados o están pendientes.
- **git add:** selecciona los cambios que se incluirán en el próximo registro.
- **git commit:** guarda un conjunto de cambios en el historial con un mensaje descriptivo.
- **git push:** envía los commits locales a un repositorio remoto.
- **git pull:** obtiene cambios del repositorio remoto y los integra en el trabajo local.
- **git clone:** copia un repositorio remoto en la computadora.
- **git branch:** permite consultar o administrar ramas de trabajo.

Una rama permite trabajar en una función o corrección sin modificar directamente la versión principal. Cuando los cambios están listos, pueden revisarse e integrarse. Es importante realizar commits claros y frecuentes para que el historial sea útil.

10. Diagramas PERT y Gantt

Diagrama PERT

PERT (Program Evaluation and Review Technique) es una técnica para planificar y analizar las actividades de un proyecto. Representa las tareas y las dependencias entre ellas, es decir, qué actividades deben completarse antes de iniciar otras. Permite estimar la duración total y reconocer el camino crítico: la secuencia de actividades que determina el tiempo mínimo necesario para terminar el proyecto. Si una tarea del camino crítico se retrasa, puede retrasarse la finalización del proyecto.

PERT resulta útil cuando hay incertidumbre en la duración de las tareas. En su enfoque clásico se pueden considerar tres estimaciones: optimista, más probable y pesimista, para calcular un tiempo esperado.

Diagrama de Gantt

El diagrama de Gantt es una herramienta de planificación que presenta las actividades en una lista y las ubica sobre una línea de tiempo mediante barras horizontales. Cada barra indica cuándo comienza una tarea, cuánto dura y cuándo termina. También puede mostrar responsables, porcentaje de avance y tareas que se realizan en simultáneo. Es útil para comunicar el cronograma y controlar el progreso del proyecto.

Diferencias y uso conjunto

PERT se concentra en las relaciones de dependencia entre tareas y permite identificar el camino crítico. Gantt se concentra en el calendario y muestra visualmente la duración y ubicación temporal de las actividades. No son herramientas opuestas: pueden utilizarse juntas. Primero se pueden analizar las dependencias con PERT y luego organizar las fechas y el avance en un Gantt.

Conclusión

Los temas de 5.º y 7.º año forman parte de un mismo proceso de desarrollo de sistemas. En 5.º se aprende a analizar problemas, representar datos y procesos, construir modelos, estudiar resultados y organizar la documentación. En 7.º se profundiza en el diseño de software mediante objetos, la administración de bases de datos, la representación de procedimientos, la programación y el trabajo colaborativo. También se incorporan herramientas para planificar y controlar proyectos. Conocer estos contenidos permite crear sistemas que respondan a las necesidades de los usuarios, estén bien organizados y puedan utilizarse y mantenerse a lo largo del tiempo.